

Build Task — Project 1: Beginner LLM App — Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through the hints in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

You're combining everything from the 5 practice exercises above into one small chatbot: memory (so it remembers earlier turns), streaming (so replies feel instant), and safe handling of the 3 failure types you already saw (bad key, too-long input, a structured-output request that doesn't come back clean).

Think of it as 3 files working together: one that talks to the model (`chat_client.py`), one that describes the shape of any structured answer you want back (`schemas.py`), and one that runs the actual terminal loop (`main.py`).

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

The Build Task is a synthesis, not new material — every function here is a small variation on something you already built in the 5 exercises above. `chat_client.py` should expose `create_client()`, `send_message(history, user_input) -> str`, and `stream_message(history, user_input)`, so `main.py` never touches the OpenAI SDK directly — it only calls your own, already-tested wrapper functions.

Reusing Doc01's `config.py` / `logging_setup.py` here isn't optional decoration — it's the whole point of building them in Doc01: this is the first of many projects that starts by importing them unchanged.

Sketch the 3-file structure and each function's signature (inputs/outputs, no bodies yet) before moving to Hint 2.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

Things to reuse, not rebuild: - `load_config()` and `get_logger()` from Doc01 — call them at the very top of `main.py`, before anything else. - The multi-turn `history` list pattern from the Intermediate exercise. - The streaming loop from the Real-world exercise. - A Pydantic model (like in Doc01's dataclass idea, but from the `pydantic` library) to describe your structured-output shape — Pydantic checks the data actually matches, instead of just hoping it does. - The specific `try/except` blocks from the Edge cases and Failure exercises — one per failure type, not one shared catch-all.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

- `from config import load_config / from logging_setup import get_logger` — same functions from Doc01, unchanged, called at the top of `main.py`.
- `history: list[dict]` — the growing message list from the Intermediate exercise; every user turn and assistant reply gets appended to it, in order.
- Streaming: reuse `stream_message()`'s loop from the Real-world exercise, but have it also return the accumulated `full_reply` string so `main.py` can append it to `history`.
- `schemas.py`: `class ExtractedInfo(BaseModel): ...` — a Pydantic model. Use `client.beta.chat.completions.parse(response_format=ExtractedInfo, ...)` (or the equivalent structured-output call for your SDK version) to get back an object Pydantic has already validated, instead of a raw string you'd have to parse and hope is valid JSON.
- 4 separate `except` blocks matching the 4 "Handles gracefully" requirements: `openai.AuthenticationError`, `openai.BadRequestError` (context length), a too-long conversation check (proactive, via `tiktoken`, before sending), and a Pydantic `ValidationError` for a structured-output reply that didn't come back clean.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

```
main.py:
    load config and logger from Doc01

    make the chat client
    start history with a system message

    loop forever:
        get user input
        try:
            stream the reply, show it as it arrives
            add both turns to history
        except bad key:
            print a clean message, stop
        except too long:
            print a clean message, drop the oldest turns, keep going

    (separately, on request) call the structured-output mode:
        try:
            get a checked object back
        except it doesn't validate:
            print a clean "couldn't understand that" message
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
chat_client.py:
def create_client() -> OpenAI: ...
def send_message(history, user_input) -> str: ...
def stream_message(history, user_input) -> str: ...    # prints as it streams,

schemas.py:
class ExtractedInfo(BaseModel): ...

main.py:
config = load_config()
logger = get_logger(__name__)
client = create_client()
history = [{"role": "system", "content": "..."}]

loop:
    user_input = input("You: ")
    try:
        reply = stream_message(history, user_input)
        history.append({"role": "user", "content": user_input})
        history.append({"role": "assistant", "content": reply})
    except openai.AuthenticationError:
        logger.error(...); print clean message; break
    except openai.BadRequestError:
        print clean message; history = history[-4:] # drop oldest turns, keep going
```

The "drop the oldest turns and keep going" line matters — a too-long conversation shouldn't kill the whole session, just force it to forget the earliest, least-relevant parts and continue.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

The trickiest part — the structured-output call and its own specific error handling:

```
from pydantic import BaseModel, ValidationError

class ExtractedInfo(BaseModel):
    name: str
    intent: str

def get_structured_reply(client, user_input):
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_input}],
            response_format=ExtractedInfo,
        )
        return completion.choices[0].message.parsed
    except ValidationError:
        return None # caller should print a clean "couldn't understand that" mes
```

Try wiring the rest (`main.py` 's loop, the auth/context-length handling) yourself before checking the Solution.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
from pydantic import BaseModel, ValidationError
from openai import OpenAI

class ExtractedInfo(BaseModel):
    name: str
    intent: str

def get_structured_reply(client: OpenAI, user_input: str) -> ExtractedInfo | None:
    try:
        completion = client.beta.chat.completions.parse(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": user_input}],
            response_format=ExtractedInfo,
        )
        return completion.choices[0].message.parsed
    except ValidationError:
        return None
```

Wire this into `main.py` behind its own command (e.g. the user types `/extract ...`), with its own `if result is None:` branch printing a clean failure message. Compare your full 3-file solution against the [Solution](#) below.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Solution

Two working approaches below. Both are correct — read both, then decide which you'd actually ship.

Simple Version

```
# chat_client.py
from openai import OpenAI

def create_client():
    return OpenAI()

def stream_message(client, history, user_input):
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(model="gpt-4o-mini", messages=history)
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

```
# main.py
import openai
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message

config = load_config()
logger = get_logger("main")
client = create_client()
history = [{"role": "system", "content": "You are a helpful assistant."}]

while True:
    user_input = input("You: ")
    try:
```

```
        stream_message(client, history, user_input)
    except openai.AuthenticationError:
        print("Login failed – check your API key.")
        break
    except openai.BadRequestError:
        print("That conversation got too long – forgetting the oldest messages.")
        history = [history[0]] + history[-4:]
```

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version — 2 approaches

Approach 1 — everything in `main.py` 's loop, minimal structure

```
# chat_client.py
from openai import OpenAI

def create_client() -> OpenAI:
    return OpenAI()

def stream_message(client: OpenAI, history: list[dict], user_input: str) -> str:
    history.append({"role": "user", "content": user_input})
    stream = client.chat.completions.create(model="gpt-4o-mini", messages=history)
    full_reply = ""
    for chunk in stream:
        piece = chunk.choices[0].delta.content
        if piece:
            print(piece, end="")
            full_reply += piece
    print()
    history.append({"role": "assistant", "content": full_reply})
    return full_reply
```

```
# schemas.py
from pydantic import BaseModel

class ExtractedInfo(BaseModel):
    name: str
    intent: str
```

```
# main.py
import sys
import openai
from pydantic import ValidationError
from config import load_config
from logging_setup import get_logger
from chat_client import create_client, stream_message
```

```

from schemas import ExtractedInfo

def main() -> None:
    config = load_config()
    logger = get_logger(__name__)
    client = create_client()
    history: list[dict] = [{"role": "system", "content": "You are a helpful assis

while True:
    user_input = input("You: ")
    if user_input == "/quit":
        break

    try:
        if user_input.startswith("/extract "):
            text = user_input.removeprefix("/extract ")
            try:
                completion = client.beta.chat.completions.parse(
                    model="gpt-4o-mini",
                    messages=[{"role": "user", "content": text}],
                    response_format=ExtractedInfo,
                )
                print(completion.choices[0].message.parsed)
            except ValidationError:
                print("Couldn't understand that clearly enough to extract str
        else:
            stream_message(client, history, user_input)
    except openai.AuthenticationError:
        logger.error("Authentication failed – check OPENAI_API_KEY.")
        print("Login failed – check your API key.")
        sys.exit(1)
    except openai.BadRequestError:
        logger.warning("Conversation exceeded context length, trimming histor
        print("That got too long – forgetting the oldest messages and continu
        history = [history[0]] + history[-4:]

if __name__ == "__main__":
    main()

```

Approach 2 — a small `ChatSession` class holding the state (more structure, easier to test)

```

# chat_client.py
from openai import OpenAI

```

```

class ChatSession:
    def __init__(self, client: OpenAI, system_prompt: str) -> None:
        self.client = client
        self.history: list[dict] = [{"role": "system", "content": system_prompt}]

    def send(self, user_input: str) -> str:
        self.history.append({"role": "user", "content": user_input})
        stream = self.client.chat.completions.create(
            model="gpt-4o-mini", messages=self.history, stream=True
        )
        full_reply = ""
        for chunk in stream:
            piece = chunk.choices[0].delta.content
            if piece:
                print(piece, end="")
                full_reply += piece
        print()
        self.history.append({"role": "assistant", "content": full_reply})
        return full_reply

    def trim(self, keep_last: int = 4) -> None:
        self.history = [self.history[0]] + self.history[-keep_last:]

```

```

# main.py
import sys
import openai
from config import load_config
from logging_setup import get_logger
from chat_client import ChatSession

def main() -> None:
    config = load_config()
    logger = get_logger(__name__)
    from openai import OpenAI
    session = ChatSession(OpenAI(), "You are a helpful assistant.")

    while True:
        user_input = input("You: ")
        try:
            session.send(user_input)
        except openai.AuthenticationError:
            logger.error("Authentication failed.")
            print("Login failed – check your API key.")
            sys.exit(1)

```

```
except openai.BadRequestError:
    print("That got too long – forgetting the oldest messages.")
    session.trim()

if __name__ == "__main__":
    main()
```

Which one should you actually use? Approach 1 is fine for a script this small. Approach 2's `ChatSession` class is worth the extra structure the moment you need more than one independent conversation at a time (e.g. a web server handling multiple users) — bundling `history` and its own `send / trim` behavior together means each session manages its own state cleanly, instead of `main.py` juggling several loose `history` lists by hand. For Project 1 specifically, either is correct — pick Approach 2 if you want practice with the pattern you'll actually need starting around Project 4.